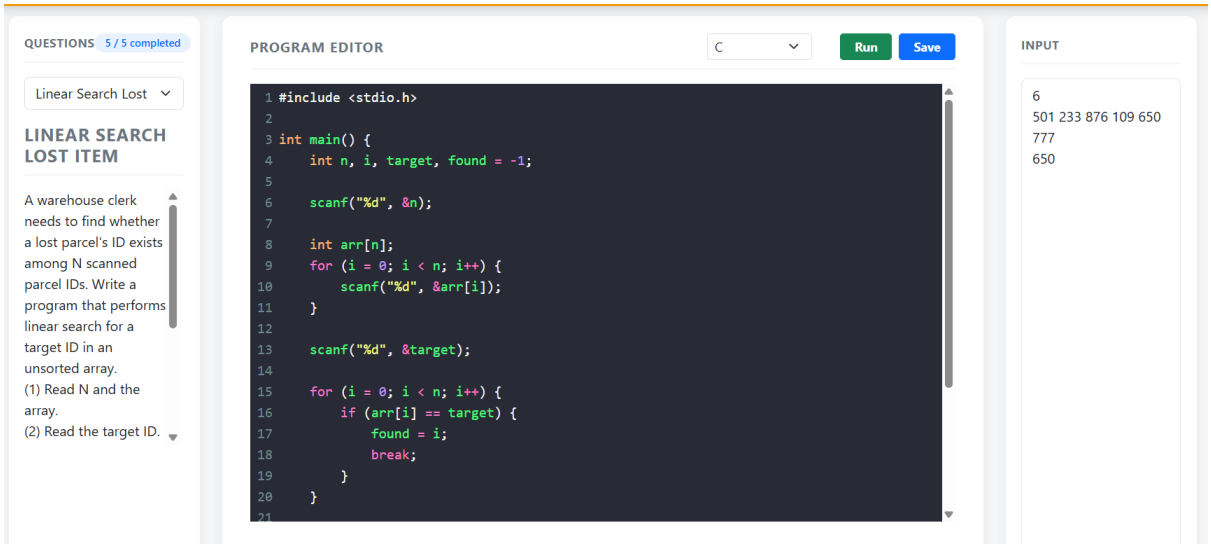
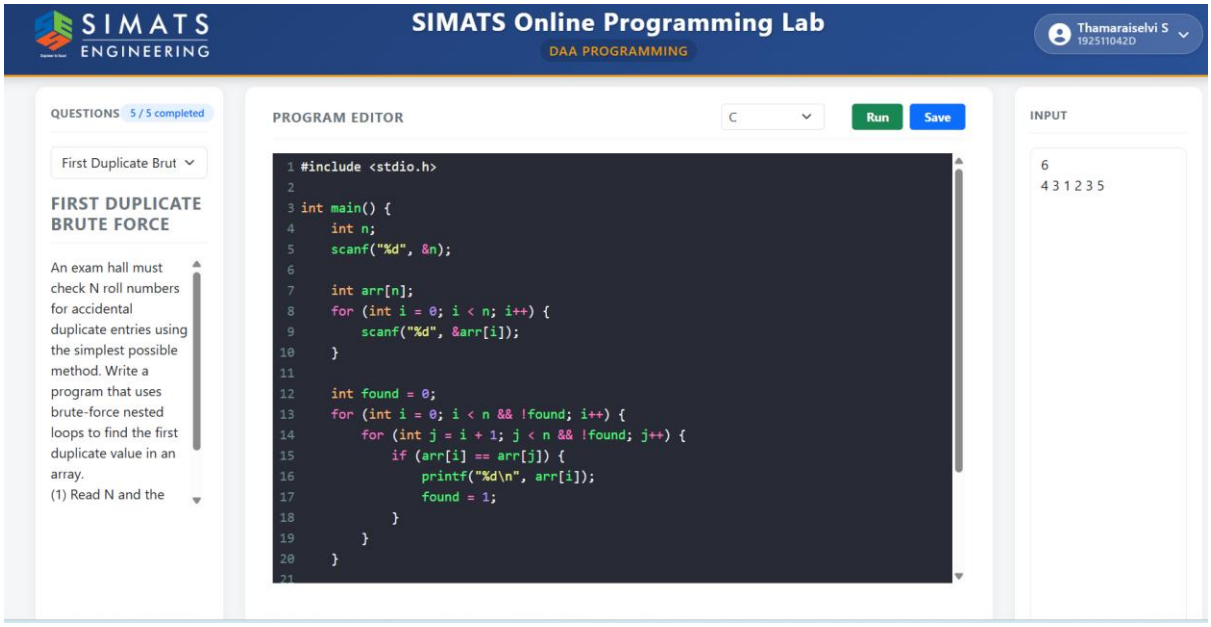
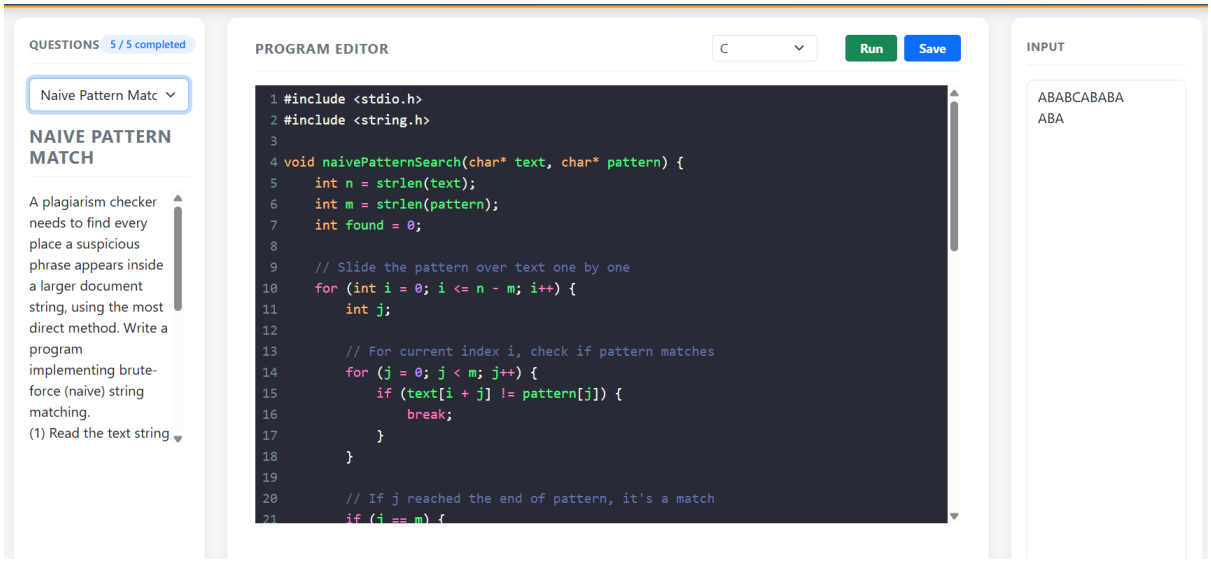
Exp 1:


QUESTIONS 5 / 5 completed
Linear Search Lost
LINEAR SEARCH LOST ITEM
A warehouse clerk needs to find whether a lost parcel's ID exists among N scanned parcel IDs. Write a program that performs linear search for a target ID in an unsorted array.
(1) Read N and the array.
(2) Read the target ID.
PROGRAM EDITOR
C
Run
Save
#include <stdio.h>
int main() {
int n, i, target, found = -1;
scanf("%d", &n);
int arr[n];
for (i = 0; i < n; i++) {
scanf("%d", &arr[i]);
}
scanf("%d", &target);
for (i = 0; i < n; i++) {
if (arr[i] == target) {
found = i;
break;
}
}
INPUT
6
501 233 876 109 650
777
650

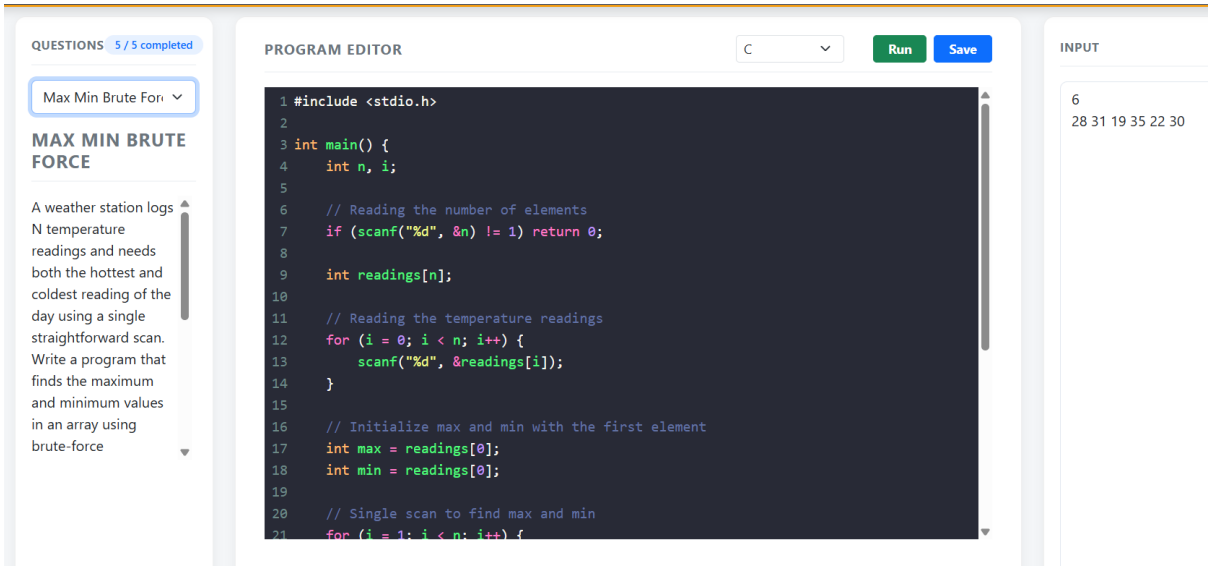
Exp 2:


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Thamaraiselvi S
192511042D
QUESTIONS 5 / 5 completed
First Duplicate Brut
FIRST DUPLICATE BRUTE FORCE
An exam hall must check N roll numbers for accidental duplicate entries using the simplest possible method. Write a program that uses brute-force nested loops to find the first duplicate value in an array.
(1) Read N and the
PROGRAM EDITOR
C
Run
Save
#include <stdio.h>
int main() {
int n;
scanf("%d", &n);
int arr[n];
for (int i = 0; i < n; i++) {
scanf("%d", &arr[i]);
}
int found = 0;
for (int i = 0; i < n && !found; i++) {
for (int j = i + 1; j < n && !found; j++) {
if (arr[i] == arr[j]) {
printf("%d\n", arr[i]);
found = 1;
}
}
}
INPUT
6
4 3 1 2 3 5

Exp 3:


QUESTIONS 5 / 5 completed
Naive Pattern Matc
NAIVE PATTERN MATCH
A plagiarism checker needs to find every place a suspicious phrase appears inside a larger document string, using the most direct method. Write a program implementing brute-force (naive) string matching.
(1) Read the text string
PROGRAM EDITOR
C
Run
Save
#include <stdio.h>
#include <string.h>
void naivePatternSearch(char* text, char* pattern) {
int n = strlen(text);
int m = strlen(pattern);
int found = 0;
// Slide the pattern over text one by one
for (int i = 0; i <= n - m; i++) {
int j;
// For current index i, check if pattern matches
for (j = 0; j < m; j++) {
if (text[i + j] != pattern[j]) {
break;
}
}
// If j reached the end of pattern, it's a match
INPUT
ABABCABABA
ABA

Exp 4:


QUESTIONS 5 / 5 completed
Max Min Brute For
MAX MIN BRUTE FORCE
A weather station logs N temperature readings and needs both the hottest and coldest reading of the day using a single straightforward scan. Write a program that finds the maximum and minimum values in an array using brute-force
PROGRAM EDITOR
C
Run
Save
#include <stdio.h>
int main() {
int n, i;
// Reading the number of elements
if (scanf("%d", &n) != 1) return 0;
int readings[n];
// Reading the temperature readings
for (i = 0; i < n; i++) {
scanf("%d", &readings[i]);
}
// Initialize max and min with the first element
int max = readings[0];
int min = readings[0];
// Single scan to find max and min
INPUT
6
28 31 19 35 22 30

Exp 5:

QUESTIONS 5 / 5 completed

Substring Existence

SUBSTRING EXISTENCE CHECK

A password strength checker wants to verify, in the simplest way, whether a known weak fragment appears anywhere inside a user's password. Write a program that checks whether a pattern string occurs as a substring of a text

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <string.h>

int main() {
    char text[100];
    char pattern[100];
    int i, j, n, m, found = 0;

    // Reading the input strings
    // Using %[^\n] to allow spaces if necessary, though simple scanf works for standa
    if (scanf("%[^\n]%*c", text) == 0) return 0;
    if (scanf("%[^\n]%*c", pattern) == 0) return 0;

    n = strlen(text);
    m = strlen(pattern);

    // Brute-force search
    for (i = 0; i <= n - m; i++) {
        for (j = 0; j < m; j++) {
            if (text[i + j] != pattern[j]) {
```

INPUT

```
mypassword123
word
```